

SQLite 실습과 연결 API

2주차 2차시 · 웹 SQL 실습
DBeaver로 원본 데이터 확인
빌드 스크립트와 main.c 함수 설명

한국어 설명 · SQL과 함수명은 원문 유지
참조: week2-2-sqlite-cli-api-v01.odp / .pdf

02

SQLITE
CLI / API

오늘의 흐름

- 01 SQL 읽기 Step 1-4 SQL의 역할을 읽습니다.
- 02 웹 실습 브라우저에서 test.db의 복사본을 조회합니다.
- 03 원본 비교 메모리 초기화 후 DBeaver 결과와 비교합니다.
- 04 소스 읽기 bootstrap.py와 main.c의 연결 생성 과정을 읽습니다.

참조 자료의 shell.c.in·legacy.c 상세 분석은 이번 범위에서 제외합니다.

참조 자료와 이번 실습의 차이

- 01 참조 자료 Python 서버와 C API, university.db를 사용합니다.
- 02 이번 실습 HTML·TypeScript와 sql.js, test.db를 사용합니다.
- 03 실습 데이터 참조 데이터는 학생 5명, 현재 배포 데이터는 학생 7명입니다.
- 04 소스 버전 소스 읽기는 참조 자료와 같은 SQLite 3.53.4를 기준으로 합니다.

웹 코드와 원본 자료의 서버 코드를 같은 실행 경로로 설명하지 않습니다.
sql.js의 버전 번호와 내부 SQLite 버전 번호도 구분합니다.

배포 파일의 위치

실습 폴더 · 파일 이름과 역할

Codes/week02-02/

test.db

sql-code/Step_1_Create.sql

sql-code/Step_2_Insert.sql

sql-code/Step_3_Queries.sql

sql-code/Step_4_Clear.sql

web/index.html

web/main.ts

web/main.js

index.html을 열고 test.db를 선택합니다. main.js는 미리 컴파일되어 있습니다.

Step 1: 테이블 생성

Step_1_Create.sql · student 테이블 발췌

```
PRAGMA foreign_keys = ON;  
CREATE TABLE IF NOT EXISTS student (  
    id INTEGER PRIMARY KEY,  
    name TEXT NOT NULL,  
    dept_name TEXT,  
    tot_cred INTEGER NOT NULL DEFAULT 0,  
    FOREIGN KEY (dept_name) REFERENCES department(dept_name)  
);
```

기본 키는 학생을 식별하고, 외래 키는 학과를 참조합니다.

dept_name은 NULL을 허용합니다. 전체 파일은 테이블 5개를 생성합니다.

Step 2: 데이터 입력

Step_2_Insert.sql · 데이터 입력 일부

```
INSERT INTO department (dept_name, building)
VALUES ('Computer Science', 'Engineering Hall');
```

```
INSERT INTO student (id, name, dept_name, tot_cred)
VALUES (1001, 'Minji Kim', 'Computer Science', 42),
       (1005, 'Doyun Jung', NULL, 0);
```

참조되는 학과를 먼저 넣고 학생을 입력합니다. 위 코드는 일부 발췌입니다.
전체 Step 2를 실행하면 기존 행을 지운 뒤 샘플 데이터를 다시 넣습니다.

기준 데이터 확인

- | | | |
|----|---------|--|
| 01 | 테이블 행수 | department 4행 / student 7행 / instructor 4행 |
| 02 | 수강 데이터 | course 5행 / enroll 9행 |
| 03 | 시작 학점 | 학생 1003의 시작 학점은 18입니다. |
| 04 | NULL 관찰 | 학생 1005의 학과는 NULL입니다. |

참조 자료의 한글 이름 대신, 현재 test.db의 영문 데이터를 그대로 사용합니다.

Step 3: 조건 조회와 NULL

Step_3_Queries.sql · 조건과 NULL

```
SELECT id, name, tot_cred FROM student  
WHERE dept_name = 'Computer Science' ORDER BY id;
```

```
SELECT id, name FROM student  
WHERE dept_name IS NULL;
```

첫 결과는 1001·1003, 두 번째 결과는 1005입니다.
NULL 여부는 = NULL이 아니라 IS NULL로 확인합니다.

Step 3: 집계와 조인

Step_3_Queries.sql · 집계와 조인

```
SELECT dept_name, AVG(tot_cred) AS avg_cred  
FROM student GROUP BY dept_name;
```

```
SELECT s.name, c.title, e.grade  
FROM student AS s  
JOIN enroll AS e ON s.id = e.student_id  
JOIN course AS c ON c.code = e.course_code  
ORDER BY s.id, c.code;
```

Mathematics의 평균 학점은 32.5, 조인 결과는 수강 내역 9행입니다.

Step 3: 변경 후 ROLLBACK

Step_3_Queries.sql · 변경과 복원

```
BEGIN TRANSACTION;  
INSERT INTO student (id, name, dept_name, tot_cred)  
VALUES (1008, 'Yejin Shin', 'Mathematics', 31);  
UPDATE student SET tot_cred = 38 WHERE id = 1003;  
SELECT id, tot_cred FROM student WHERE id = 1003;  
DELETE FROM student WHERE id = 1008;  
ROLLBACK;
```

트랜잭션 블록을 함께 실행합니다. ROLLBACK 전에는 38, 이후에는 18입니다.
현재 실습에서는 브라우저 메모리 안에서만 변경을 관찰합니다.

Step 4: 데이터 비우기

Step_4_Clear.sql · 전체 데이터 삭제

```
PRAGMA foreign_keys = ON;  
BEGIN TRANSACTION;  
DELETE FROM enroll;  
DELETE FROM course;  
DELETE FROM instructor;  
DELETE FROM student;  
DELETE FROM department;  
COMMIT;
```

자식 테이블부터 비워 외래 키 충돌을 피합니다. 테이블 구조는 남습니다.
Step 2로 다시 입력하거나 웹의 Reset memory로 복원합니다.

웹 실습: 파일을 열고 SQL 실행

- 01 웹페이지 열기 `web/index.html`을 브라우저에서 엽니다.
- 02 DB 파일 선택 SQLite ready가 표시되면 `test.db`를 선택합니다.
- 03 조회 선택 `student` 또는 `All students`를 선택합니다.
- 04 SQL 실행 Run SQL을 눌러 결과를 확인합니다.

인터넷 연결은 `sql.js`와 WASM 파일을 불러올 때 필요합니다.
선택한 DB를 서버로 전송하거나 원본 파일에 저장하지 않습니다.

HTML과 TypeScript의 역할

- 01 index.html 파일 선택, SQL 입력, 결과 표시 영역
- 02 main.ts 파일 읽기와 SQLite 실행을 담당하는 소스
- 03 main.js 브라우저가 실제로 실행하는 컴파일 결과
- 04 sql.js WebAssembly로 SQLite를 실행하는 라이브러리

이번 웹 예제는 서버 없이 실행합니다. 참조 자료의 server.py는 사용하지 않습니다.

TypeScript: 파일을 메모리 DB로 열기

web/main.ts · 파일 읽기와 메모리 DB 생성

```
const bytes = new Uint8Array(await file.arrayBuffer());  
openMemoryDatabase(bytes);  
originalBytes = bytes;
```

```
// Inside openMemoryDatabase:  
const candidate = new runtime.Database(bytes.slice());  
candidate.exec("PRAGMA foreign_keys = ON;");
```

arrayBuffer()로 파일의 바이트를 읽어 메모리 DB를 만듭니다.
originalBytes를 보관하므로 Reset memory로 처음 상태를 복원할 수 있습니다.

TypeScript: 실행 결과를 표로 표시

web/main.ts · 실행 결과의 전달

```
const results = database.exec(editor.value);  
renderResults(results);  
refreshTables();
```

```
// A result contains:  
// columns: column names  
// values: row values
```

exec()는 SQL을 실행하고 결과 집합을 반환합니다.
표시는 textContent를 사용하며 NULL과 빈 문자열을 구분합니다.

메모리 변경과 원본 파일 비교

브라우저에서 실행할 SQL

```
UPDATE student SET tot_cred = 38 WHERE id = 1003;  
SELECT id, tot_cred FROM student WHERE id = 1003;
```

웹에서 실행: 38 / DBeaver에서 SELECT만 실행: 18

웹에서 Reset memory 후 다시 조회: 18

이 예제는 ROLLBACK 없이 실행하므로 초기화 효과를 직접 볼 수 있습니다.

DBeaver: 기존 test.db 연결

- 01 새 연결 Database > New Database Connection을 선택합니다.
- 02 DB 파일 SQLite를 선택하고 기존 test.db 파일을 지정합니다.
- 03 연결 완료 필요하면 드라이버를 내려받고 연결을 완료합니다.
- 04 데이터 확인 Tables에서 student를 열어 데이터를 확인합니다.

SQL Editor에서 같은 SELECT를 실행해 웹 결과와 비교합니다.
이번 비교에서는 DBeaver로 원본을 수정하거나 Step 4를 실행하지 않습니다.

CLI와 SQL 입력창의 구분

CLI 점 명령과 SQL의 비교

```
.tables  
.schema student  
.databases
```

```
SELECT name FROM sqlite_schema  
WHERE type = 'table';
```

위의 점 명령 3개는 CLI 전용입니다. 웹 SQL 창에서는 아래 SELECT를 사용합니다.
CLI 내부 함수의 상세 추적은 이번 설명에서 제외합니다.

전체 소스 트리에서 먼저 볼 디렉터리

- 01 src/ SQLite 엔진 구현. 이번에는 main.c의 연결 API를 읽습니다.
- 02 tool/ 소스 생성, 빌드와 진단을 돕는 도구·스크립트.
- 03 ext/ FTS, R-tree, ICU, WASM 등 확장 기능별 소스.
- 04 doc/ 빌드 방법과 내부 구현 설명. 전체 사용자 문서는 별도 관리.

확인한 폴더: ~/Downloads/sqlite-src-3530400

이 폴더는 전체 원본입니다. sqlite3.c는 여러 소스를 합쳐 생성하는 빌드 결과입니다.

테스트·빌드 지원 디렉터리

- 01 test/ · mptest/ test는 기능·회귀 테스트, mptest는 다중 프로세스 테스트.
- 02 autotest/ configure가 사용하는 설정 도구와 SQLite용 검사 로직.
- 03 autoconf/ amalgamation 배포 패키지의 설정·빌드 지원 파일.
- 04 art/ · contrib/ art는 로고·아이콘, contrib는 부가 기여 자료.

compat/는 이 배포본에서 비어 있습니다. 현재 분석할 엔진 코드는 없습니다.
이번 수업은 이 폴더들의 역할만 구분하고 세부 테스트·확장 분석은 생략합니다.

configure에서 main.mk까지

- 01 configure 시작용 셸 스크립트. autoseup의 실행기로 넘깁니다.
- 02 auto.def SQLite의 옵션·컴파일러·시스템 의존성 검사 정의.
- 03 Makefile.in 검사 결과와 경로를 채워 Makefile을 만드는 템플릿.
- 04 main.mk 소스 생성·컴파일·설치·테스트의 공통 규칙. Makefile이 포함.

진행 순서: configure 실행, 환경 설정, Makefile 생성, make로 빌드.
main.mk는 필요한 변수를 설정한 상위 Makefile에서 포함해 사용하는 파일입니다.

나머지 루트 파일의 역할

- 01 대체 빌드 Makefile.msc는 MSVC용, Makefile.linux-generic은 대체 설정용.
- 02 make.bat Windows에서 nmake /f Makefile.msc로 전달하는 래퍼.
- 03 버전·이력 VERSION은 버전, manifest 계열은 Fossil 소스 이력 정보.
- 04 안내·라이선스 README.md는 빌드·구조 안내, LICENSE.md는 라이선스 안내.

sqlite.pc.in·sqlite3.pc.in: pkg-config 메타데이터 템플릿
sqlite3.1: CLI 매뉴얼 / magic.txt: 파일 형식 식별 규칙

이 소스 트리에서 직접 빌드한다면

macOS / Linux 예시 · 별도 빌드 디렉터리 사용

```
cd ~/Downloads
mkdir -p sqlite-build-3530400
cd sqlite-build-3530400
../sqlite-src-3530400/configure
make sqlite3
./sqlite3 :memory: "SELECT sqlite_version();"
```

강의용 bootstrap.py는 배포본 검증과 컴파일을 묶은 별도 자동화입니다.
이 명령은 공식 소스 트리의 빌드 경로를 설명하는 예시이며, 이번 작업에서 실행하지 않았습니다.

빌드할 파일과 읽을 파일은 다릅니다

- 01 `sqlite3.c · shell.c` amalgamation 배포본. 참조 프로젝트의 컴파일 입력.
- 02 `bootstrap.py` 소스 검증, 컴파일, 버전 확인과 DB 준비를 자동화.
- 03 `src/main.c` 공개 연결 API와 초기화·연결 생성 구현.
- 04 이번 분석 범위 빌드 스크립트와 연결 생성까지. 실행 엔진 내부는 제외.

분석 기준: `vendor/sqlite-src-3530400/src` / 파일명과 함수명을 함께 기록하세요.

빌드 스크립트는 준비 과정을 자동화합니다

- 01 배포본 확인 `unpack_verified()`: SHA3-256 확인 후 압축 해제.
- 02 컴파일 공유 라이브러리, `sqlite3 CLI`, `api_demo`를 생성.
- 03 버전 확인 빌드한 라이브러리의 `version`과 `source ID` 비교.
- 04 DB 준비 `university.db`가 없으면 생성하고, 있으면 보존.

참조 프로젝트에서 `python3 bootstrap.py`로 실행합니다.
현재 브라우저 메모리 실습에서는 이 빌드 과정이 필요하지 않습니다.

main.c: 공개 API는 내부 구현으로 위임

src/main.c:3729-3736 · 주석 생략 / 줄바꿈 조정

```
int sqlite3_open_v2(
    const char *filename,
    sqlite3 **ppDb,
    int flags,
    const char *zVfs
){
    return openDatabase(filename, ppDb,
                        (unsigned int)flags, zVfs);
}
```

filename: 파일명 / ppDb: 출력 핸들 / flags: 열기 옵션 / zVfs: VFS 이름

openDatabase는 연결을 단계적으로 구성

- 01 초기화 · :3376-3379 출력 핸들을 비우고 sqlite3_initialize 호출.
- 02 연결 구조체 · :3422 sqlite3 객체 할당 → 옵션·기본 상태 설정.
- 03 파일 계층 URI/VFS(:3596) → B-tree 연결(:3612)
- 04 종료 경로 · :3692 opendb_out에서 결과와 오류를 정리해 반환.

이번에는 각 단계의 역할과 경계만 읽습니다. VFS·B-tree 내부는 이후 실습에서 다룹니다.

초기화 뒤에 연결 구조체를 만듭니다

src/main.c:3376-3380, 3422-3423 · 중간 생략

```
*ppDb = 0;
#ifdef SQLITE_OMIT_AUTOINIT
rc = sqlite3_initialize();
if( rc ) return rc;
#endif

/* ... */
db = sqlite3MallocZero( sizeof(sqlite3) );
if( db==0 ) goto opendb_out;
```

출력 핸들을 먼저 비웁니다. 자동 초기화에 성공하면 연결 객체를 할당합니다.
초기화 실패는 즉시 반환하고, 할당 실패는 opendb_out으로 이동합니다.

종료 경로에서 상태와 핸들을 정리합니다

src/main.c:3700-3706 · 실제 원본 발췌

```
if( (rc&0xff)==SQLITE_NOMEM ){
    sqlite3_close(db);
    db = 0;
}else if( rc!=SQLITE_OK ){
    db->openState = SQLITE_STATE_SICK;
}
*ppDb = db;
```

메모리 부족이면 연결을 닫고 NULL을 돌려줍니다. 다른 오류에서는 핸들이 남을 수 있습니다. 호출자는 반환 코드 rc를 확인하고, 남아 있는 핸들을 sqlite3_close()로 정리합니다.

추적 활동: 파일 · 함수 · 행으로 답하기

- 01 질문 1 bootstrap.py는 어떤 파일로 CLI를 빌드하나요?
- 02 질문 2 sqlite3_open_v2의 네 인자와 위임 대상을 쓰세요.
- 03 질문 3 openDatabase의 초기화·할당·종료 위치를 찾으세요.
- 04 질문 4 실패한 open 호출에도 연결 정리가 필요한 이유는?

검색 도구: 편집기의 찾기 또는 rg / 답에는 버전, 파일, 함수 이름을 함께 적으세요.

수업 마무리

- 01 웹 실습 웹은 선택한 DB의 메모리 복사본을 사용합니다.
- 02 초기화 Step 4는 데이터를 비우고 Reset memory는 원래 상태를 복원합니다.
- 03 빌드 스크립트 bootstrap.py는 빌드와 버전 검증을 자동화합니다.
- 04 main.c 분석 main.c에서는 연결 API의 인자, 반환값, 정리 책임을 읽습니다.

확인 질문: 웹에서 바꾼 학점이 DBeaver에는 왜 그대로일까요?

확인 질문: sqlite3_open_v2()의 반환값과 ppDb는 어떻게 다를까요?